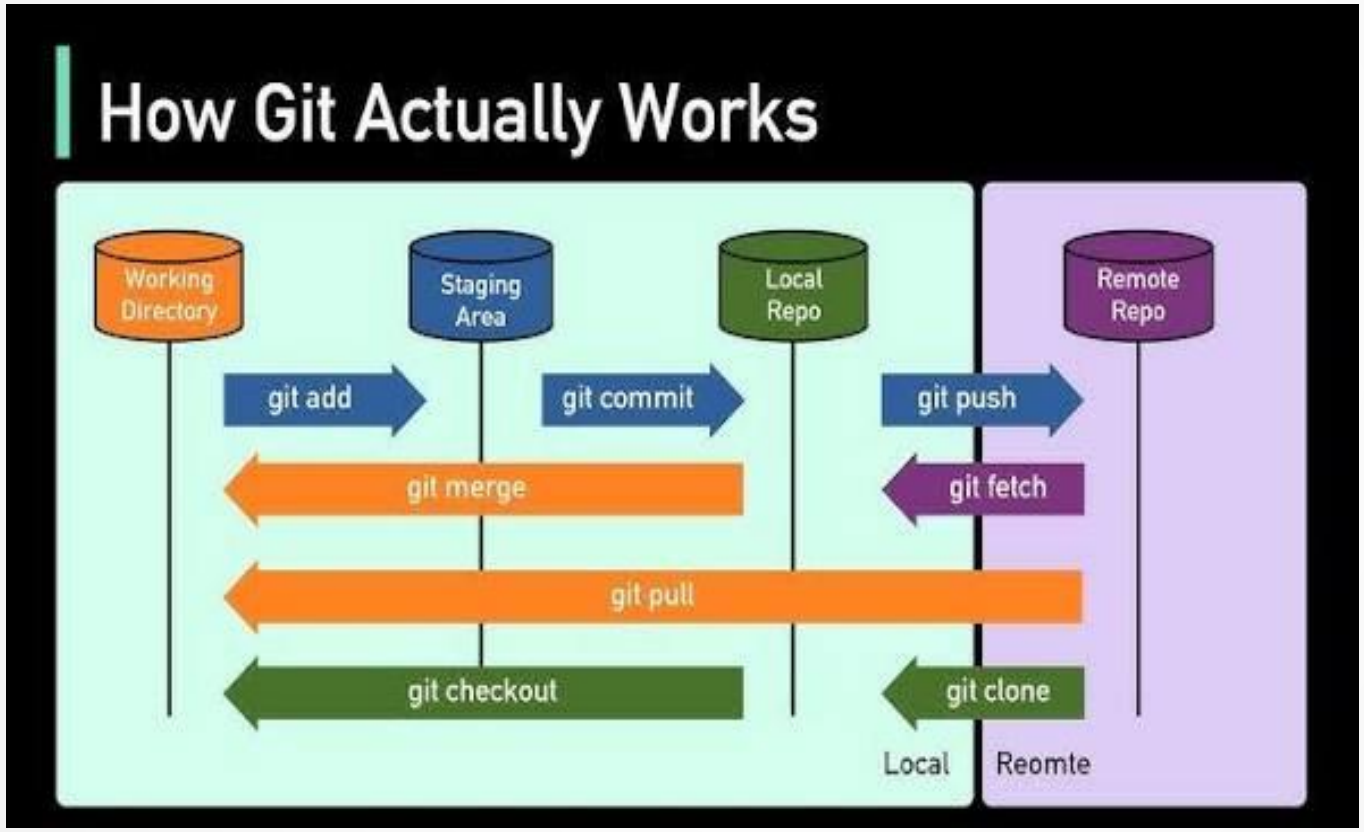
Git, GitHub & GitLab

Presented by Nithish D

What is Version control

- **Version control is a system that records changes to your files over time so you can review or bring back specific versions later.** It acts like a time machine for your digital work, making sure you never lose an old draft or accidentally ruin a project.
- **It Stops Messy Folders:** You no longer need to clog up your computer with duplicate files.
- **It Is a Safety Net:** If you delete a whole page by accident or your file gets corrupted, you do not have to start over. You just load the previous snapshot.
- **It Tracks Progress:** The system remembers exactly who made a change, what they changed, and when they did it.

Git Architecture



Clone & Commit

- **Clone:** This downloads a complete copy of an existing project from the cloud (like GitHub) onto your local computer.

  Ex: git clone <url-of-the-cloud-project>

- **Commit:** This takes a snapshot of the changes you made on your computer and locks them into your local history log.

  First, tell Git which modified files you want to include in the snapshot:
  Ex: git add <file-name>

  Next, lock in the snapshot with a short descriptive message:
  Ex: git commit -m "Fixed the typo on the homepage"

Push & Pull

- **Push:** This sends all your local, saved commits up to the cloud repository so your team can see them.

  Ex: git push origin main
- **Pull:** This fetches any new changes your teammates uploaded to the cloud and merges them into your computer's files.

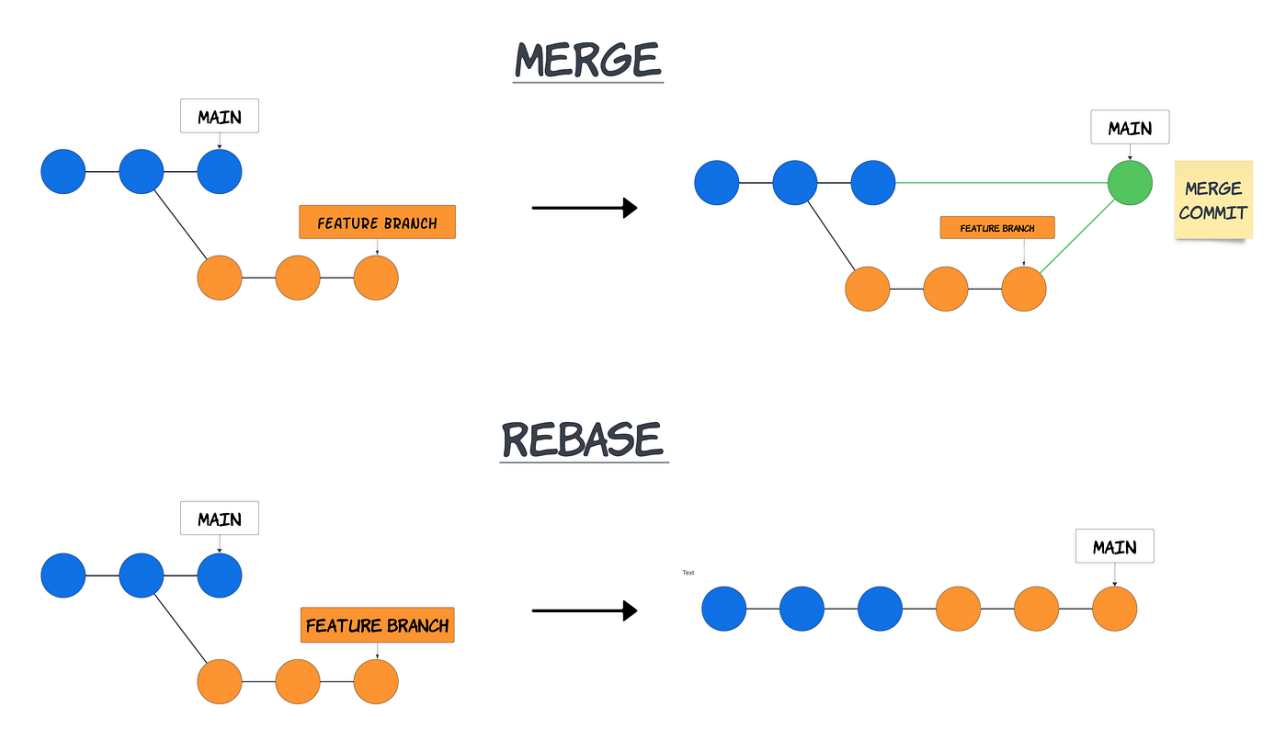
  Ex: git pull origin main
- **Fetch:** The git fetch downloads the newest code updates from the online repository to your computer.
  It keeps your personal files safe by not changing any of your current work.

  Ex: git fetch origin main

Branching stratergy

- A branching strategy is a set of rules that tells a team how to split their work into separate tracks so they don't mess up the main project.
- Instead of everyone editing the master file at the same time, Git lets you duplicate the project instantly into a safe sidebar track called a "Branch”.
- You can write messy code, test features, and break things on your branch. The main project stays completely safe and untouched.
- **The Merge**: Once your new feature works perfectly, you safely "merge" your branch's changes back into the main project.

Merging & Rebase



Merging

- Merging takes the changes from one branch and combines them into another by creating a special "tie-together" checkpoint called a **Merge Commit.**
- **The Good**: It is 100% safe. It leaves your history alone and records the exact truth of when work happened.
- **The Bad**: If a big team merges files constantly, the history log turns into a messy bowl of tangled spaghetti.
- The Commands:
  git checkout main
  git merge feature-login

Rebase

- Rebasing lifts your entire branch up, moves its starting point to the very tip of the main branch, and drops your changes back down. It completely changes the past.
- It looks like a perfectly straight line. The side branch vanishes, and your work is placed neatly in front of your teammate's work.
- **The Good**: It keeps your project history incredibly clean, straight, and easy to read.
- **The Bad**: It dangerous if you share your branch with others online. Rewriting history that other people are actively working on will break their code.
- The Commands:
  git checkout feature-login
  git rebase main

Merge Requests

- **How Merge Request Works in 4 Steps**

1. **You Push Your Work**: You finish your feature on your private branch and push it up to the cloud.
2. **You Open the Request**: You click a button online to open a PR/MR. You write a short summary of what you built.
3. **The Team Reviews It**: Your teammates look at the code. They can say "Looks good!" or ask you to fix a few lines.
4. **The Merge**: Once approved, someone clicks the **Merge** button. Your changes are officially added to the main project branch.

GitHub and GitLab

GitHub	GitLab
▶ GitHub is the most widely used platform in the world. It is the go-to home for open-source code and the global developer community. ▶ Community and ecosystem. It is incredibly easy to find, share, and contribute to public projects. ▶ **GitHub Actions** (a built-in tool that automatically tests your code to make sure it works every time you upload it)	▶ GitLab is a major competitor that focuses on providing absolutely everything a software team needs inside a single platform. ▶ "DevOps" integration. GitLab does not just host your code; it tracks planning, security testing, deployment, and monitoring all under one roof. ▶ **Built-in CI/CD.** It automates the entire process of testing code and launching it live to users

Access Management

- Access management controls who can enter your repository (project folder) and what actions they are allowed to take. Instead of giving everyone full control, you assign specific roles.
- **Common Roles**:

1. **Read-Only / Guest**: Can view the code and download it, but cannot change anything. (Great for clients or junior learners).
2. **Developer / Contributor**: Can create side branches and write code, but cannot touch the main production branch without approval.
3. **Maintainer / Owner**: Has total control. Can delete the project, change settings, and bypass all rules.

Branch Policies

- Branch policies (often called "Protected Branches") are strict rules applied to your most important branches, like main. They stop developers from uploading code directly to the main track.
- **Popular Branch Policies:**

1. **Require a Pull/Merge Request:** No one can upload directly to main. You *must* create a branch and ask for a review first.
2. **Require Approvals:** At least one or two senior team members must look at your Pull Request and click "Approve" before the code can be merged.
3. **Require Passing Tests:** Automated status checks must run and prove that your code doesn't break the application before the merge button unlocks.
4. **Restrict Deletions:** Prevents anyone (even by accident) from deleting the main branch and erasing the company's core product.